

AEGIS

Autonomous Trading Intelligence System

V1.2

ARCHITECTURE FROZEN

Study / Sandbox / Paper Trading
2026

1. Executive Summary

O Aegis V1.2 é uma plataforma modular de estudo para swing trading autônomo, utilizando IA para análise e geração de decisões estruturadas, com execução exclusivamente em Replay e Paper Trading.

A arquitetura prioriza segurança, determinismo, rastreabilidade, experimentação e velocidade de desenvolvimento. Python foi escolhido como runtime principal por sua integração com IA, dados quantitativos e velocidade de iteração.

A V1.2 não possui execução financeira LIVE. Uma futura integração com broker deve ser tratada como novo escopo e passar por nova revisão arquitetural.

Status: ARCHITECTURE FROZEN

2. Escopo da V1.2

- Análise de mercado assistida por LLM.
- Geração de decisões estruturadas.
- Risk Engine determinístico.
- Replay histórico determinístico.
- Paper Trading.
- Portfolio e accounting.
- Dashboard operacional.
- Configuração de providers e modelos LLM.
- Auditoria ponta a ponta.
- Observabilidade.
- Kill switch.

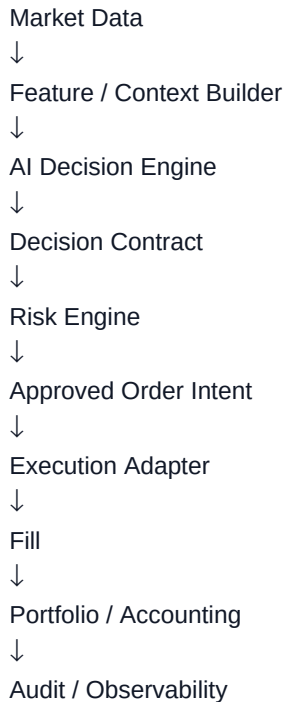
FORA DO ESCOPO: execução LIVE, conexão com dinheiro real, HFT, otimização prematura de baixa latência e qualquer integração financeira de produção.

Escopo acadêmico: completamente removido da V1.2.

3. Visão Arquitetural

AEGIS é um sistema modular de análise e execução autônoma para swing trading em ambiente de estudo/sandbox. A inteligência artificial é responsável por análise e geração de decisões estruturadas, mas não possui autoridade direta para executar ordens.

O fluxo canônico é:



O mesmo pipeline deve poder ser alimentado por Replay e Paper.

3.1 Modos Operacionais

REPLAY:

Executa datasets históricos de forma determinística e sem execução financeira.

PAPER:

Executa através do simulador de execução, sem dinheiro real.

LIVE:

Arquiteturalmente previsto através de adapter, porém NÃO implementado, NÃO habilitado e NÃO permitido na V1.2.

3.2 Princípios Arquiteturais

- Risk Engine possui autoridade superior à decisão da IA.
- LLM nunca chama broker diretamente.
- Somente candles fechados podem alimentar decisões.
- Não existe look-ahead em Replay.
- Hard limits não podem ser sobrescritos pela IA.
- Toda operação crítica deve ser rastreável.
- Falha de LLM deve resultar em comportamento seguro.

- Secrets nunca devem aparecer no frontend ou logs.
- Replay deve ser determinístico quando executado em modo determinístico.
- Toda regra crítica deve possuir teste automatizado quando tecnicamente possível.
- Frontend não contém regras críticas de risco.
- Nenhuma funcionalidade LIVE faz parte da V1.2.

4. Stack Tecnológica Congelada

Camada	Tecnologia
Runtime	Python 3.12+
API	FastAPI
Frontend	React + TypeScript + Vite
Database	PostgreSQL
ORM	SQLAlchemy 2
Migrations	Alembic
Cache / messaging	Redis, somente quando necessário
Data / Quant	Polars / Pandas
Testing	pytest
Containers	Docker + Docker Compose
LLM	Provider abstraction
Observability	Structured logs + metrics + audit trail

Decisão de performance: a V1.2 não é otimizada para baixa latência. O objetivo é velocidade de desenvolvimento, determinismo, segurança, testabilidade e facilidade de experimentação. Componentes críticos podem ser extraídos futuramente caso exista evidência objetiva de gargalo.

5. Segurança e Controle de Risco

- A IA não possui autoridade direta para executar ordens.
- Toda intenção de ordem deve passar pelo Risk Engine.
- Hard limits são determinísticos.
- Kill switch bloqueia novas execuções.
- Falhas de provider LLM resultam em comportamento seguro.
- O frontend não possui autoridade para alterar regras críticas.
- Secrets são mantidos fora do código e fora do frontend.
- LIVE execution não existe na V1.2.

A regra central é: LLM pode recomendar; Risk Engine pode rejeitar; Execution Adapter somente executa uma intenção previamente aprovada.

6. Plano de Implementação

A implementação foi dividida em 16 fases. Cada fase possui objetivo, visão arquitetural, critérios de aceite e auditoria específica. Uma fase só é considerada concluída após o respectivo gate.

FASE 01 — Foundation, Stack & Repository

Objetivo

Estabelecer a fundação executável do Aegis, congelar a stack tecnológica, criar a estrutura do monorepo, configuração por ambiente, Docker Compose, linting, testes e health checks.

Visão Arquitetural

Python é o runtime oficial do backend. FastAPI expõe a API. React/TypeScript é o frontend. PostgreSQL é a persistência principal. O projeto deve separar domínio, infraestrutura, interfaces e adapters. Nenhum broker real deve ser introduzido nesta fase.

Critérios de Aceite

- Docker Compose sobe todos os serviços previstos sem erro.
- Endpoint /health retorna HTTP 200.
- pytest executa sem erro estrutural.
- Configuração utiliza variáveis de ambiente e não secrets hardcoded.
- Frontend e backend possuem build reproduzível.
- Nenhum componente de execução LIVE é criado.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 1 DO PROJETO AEGIS V1.2.

FASE:
Foundation, Stack & Repository

OBJETIVO:
Estabelecer a fundação executável do Aegis, congelar a stack tecnológica,
criar a estrutura do monorepo, configuração por ambiente, Docker Compose,
linting, testes e health checks.

REGRAS ARQUITETURAIS:
Python é o runtime oficial do backend. FastAPI expõe a API. React/TypeScript
é o frontend. PostgreSQL é a persistência principal. O projeto deve separar
domínio, infraestrutura, interfaces e adapters. Nenhum broker real deve ser
introduzido nesta fase.

CRITÉRIOS DE ACEITE:
- Docker Compose sobe todos os serviços previstos sem erro.
- Endpoint /health retorna HTTP 200.
- pytest executa sem erro estrutural.
- Configuração utiliza variáveis de ambiente e não secrets hardcoded.
- Frontend e backend possuem build reproduzível.
- Nenhum componente de execução LIVE é criado.

INSTRUÇÕES OBRIGATÓRIAS:
1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
```

- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 1.

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 1 DO AEGIS V1.2.

FASE:

Foundation, Stack & Repository

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Verificar exatamente a stack congelada.
- Verificar ausência de tecnologia alternativa introduzida sem autorização.
- Verificar Docker/Compose.
- Verificar isolamento de secrets.
- Executar testes e health checks.
- Confirmar ausência de broker LIVE.

CRITÉRIOS DE ACEITE:

- Docker Compose sobe todos os serviços previstos sem erro.
- Endpoint /health retorna HTTP 200.
- pytest executa sem erro estrutural.
- Configuração utiliza variáveis de ambiente e não secrets hardcoded.
- Frontend e backend possuem build reproduzível.
- Nenhum componente de execução LIVE é criado.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 02 — Domain Model & Contracts

Objetivo

Criar os contratos centrais e modelos de domínio que serão compartilhados pelos demais módulos: MarketData, Candle, Signal, Decision, Order, Position, Fill, Portfolio, RiskDecision e eventos de auditoria.

Visão Arquitetural

Os contratos são a fonte de verdade entre módulos. A camada LLM nunca retorna diretamente uma ordem. O fluxo deve passar por Decision Contract, Risk Engine e Execution Adapter.

Critérios de Aceite

- Todos os objetos críticos possuem schema explícito.
- Validação rejeita payloads inválidos.
- Timestamps são timezone-aware.
- Enums são usados para estados e ações críticas.
- Contratos possuem testes unitários.
- Nenhuma decisão de IA pode representar diretamente uma ordem executável.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 2 DO PROJETO AEGIS V1.2.

FASE:
Domain Model & Contracts

OBJETIVO:
Criar os contratos centrais e modelos de domínio que serão compartilhados
pelos demais módulos: MarketData, Candle, Signal, Decision, Order,
Position, Fill, Portfolio, RiskDecision e eventos de auditoria.

REGRAS ARQUITETURAIS:
Os contratos são a fonte de verdade entre módulos. A camada LLM nunca
retorna diretamente uma ordem. O fluxo deve passar por Decision Contract,
Risk Engine e Execution Adapter.

CRITÉRIOS DE ACEITE:
- Todos os objetos críticos possuem schema explícito.
- Validação rejeita payloads inválidos.
- Timestamps são timezone-aware.
- Enums são usados para estados e ações críticas.
- Contratos possuem testes unitários.
- Nenhuma decisão de IA pode representar diretamente uma ordem executável.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 2.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 2 DO AEGIS V1.2.

FASE:

Domain Model & Contracts

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Verificar schemas e enums.
- Testar payloads inválidos.
- Verificar timestamps timezone-aware.
- Verificar separação Decision → Risk → Execution.
- Confirmar que nenhum contrato permite bypass do Risk Engine.

CRITÉRIOS DE ACEITE:

- Todos os objetos críticos possuem schema explícito.
- Validação rejeita payloads inválidos.
- Timestamps são timezone-aware.
- Enums são usados para estados e ações críticas.
- Contratos possuem testes unitários.
- Nenhuma decisão de IA pode representar diretamente uma ordem executável.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 03 — Market Data

Objetivo

Implementar ingestão, normalização, validação e armazenamento de dados de mercado para o ambiente de estudo.

Visão Arquitetural

Market Data é um adapter substituível. O domínio não deve conhecer detalhes do provider. Dados devem ser normalizados antes de entrarem no pipeline.

Critérios de Aceite

- Candles possuem timestamp consistente e timezone-aware.
- Duplicatas são detectadas/rejeitadas.
- Dados futuros são rejeitados.
- Gaps são detectáveis.
- Símbolos possuem normalização consistente.
- Origem e versão do dataset são rastreáveis.
- Testes cobrem dados inválidos.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 3 DO PROJETO AEGIS V1.2.

FASE:
Market Data

OBJETIVO:
Implementar ingestão, normalização, validação e armazenamento de dados
de mercado para o ambiente de estudo.

REGRAS ARQUITETURAIS:
Market Data é um adapter substituível. O domínio não deve conhecer detalhes
do provider. Dados devem ser normalizados antes de entrarem no pipeline.

CRITÉRIOS DE ACEITE:
- Candles possuem timestamp consistente e timezone-aware.
- Duplicatas são detectadas/rejeitadas.
- Dados futuros são rejeitados.
- Gaps são detectáveis.
- Símbolos possuem normalização consistente.
- Origem e versão do dataset são rastreáveis.
- Testes cobrem dados inválidos.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 3.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 3 DO AEGIS V1.2.

FASE:

Market Data

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Testar candle futuro.
- Testar candle duplicado.
- Testar timestamp inválido.
- Testar gap.
- Verificar rastreabilidade do provider.
- Confirmar que dados inválidos não são silenciosamente corrigidos.

CRITÉRIOS DE ACEITE:

- Candles possuem timestamp consistente e timezone-aware.
- Duplicatas são detectadas/rejeitadas.
- Dados futuros são rejeitados.
- Gaps são detectáveis.
- Símbolos possuem normalização consistente.
- Origem e versão do dataset são rastreáveis.
- Testes cobrem dados inválidos.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 04 — Deterministic Replay

Objetivo

Criar o primeiro mecanismo de replay determinístico para executar o pipeline de decisão sobre datasets históricos sem acesso a informações futuras.

Visão Arquitetural

Replay deve possuir interface própria e alimentar o mesmo contrato de dados utilizado pelo restante do sistema. O replay é uma fundação de validação, não uma funcionalidade posterior.

CrITÉrios de Aceite

- Dataset pode ser reproduzido do início ao fim.
- Execuções com a mesma entrada produzem a mesma sequência de eventos.
- Somente candles disponíveis naquele ponto temporal são visíveis.
- Não existe look-ahead.
- Execução registra seed/configuração quando aplicável.
- Replay possui testes de determinismo.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 4 DO PROJETO AEGIS V1.2.

FASE:
Deterministic Replay

OBJETIVO:
Criar o primeiro mecanismo de replay determinístico para executar o
pipeline de decisão sobre datasets históricos sem acesso a informações futuras.

REGRAS ARQUITETURAIS:
Replay deve possuir interface própria e alimentar o mesmo contrato de dados
utilizado pelo restante do sistema. O replay é uma fundação de validação,
não uma funcionalidade posterior.

CRITÉRIOS DE ACEITE:
- Dataset pode ser reproduzido do início ao fim.
- Execuções com a mesma entrada produzem a mesma sequência de eventos.
- Somente candles disponíveis naquele ponto temporal são visíveis.
- Não existe look-ahead.
- Execução registra seed/configuração quando aplicável.
- Replay possui testes de determinismo.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 4.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 4 DO AEGIS V1.2.

FASE:

Deterministic Replay

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Executar o mesmo replay duas vezes e comparar eventos.
- Introduzir candle futuro e verificar rejeição.
- Verificar que o pipeline não acessa dados posteriores.
- Verificar isolamento entre Replay e provider LIVE.
- Verificar persistência da configuração do replay.

CRITÉRIOS DE ACEITE:

- Dataset pode ser reproduzido do início ao fim.
- Execuções com a mesma entrada produzem a mesma sequência de eventos.
- Somente candles disponíveis naquele ponto temporal são visíveis.
- Não existe look-ahead.
- Execução registra seed/configuração quando aplicável.
- Replay possui testes de determinismo.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 05 — AI / Decision Engine

Objetivo

Implementar a camada de análise por IA, incluindo provider abstraction, prompt/context builder, parsing, validação e geração do Decision Contract.

Visão Arquitetural

O LLM é um componente de análise, não de execução. Toda saída deve passar por schema validation. Provider, modelo, versão de prompt, contexto e timestamp devem ser registrados.

CrITÉrios de Aceite

- Provider LLM é abstraído por interface.
- Resposta inválida é rejeitada.
- Confidence é validada no intervalo definido.
- Prompt/model/version são auditáveis.
- Somente candles fechados entram no contexto.
- LLM não possui acesso direto ao Execution Adapter.
- Existe fallback seguro para erro de provider.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 5 DO PROJETO AEGIS V1.2.

FASE:
AI / Decision Engine

OBJETIVO:
Implementar a camada de análise por IA, incluindo provider abstraction,
prompt/context builder, parsing, validação e geração do Decision Contract.

REGRAS ARQUITETURAIS:
O LLM é um componente de análise, não de execução. Toda saída deve passar
por schema validation. Provider, modelo, versão de prompt, contexto e
timestamp devem ser registrados.

CRITÉRIOS DE ACEITE:
- Provider LLM é abstraído por interface.
- Resposta inválida é rejeitada.
- Confidence é validada no intervalo definido.
- Prompt/model/version são auditáveis.
- Somente candles fechados entram no contexto.
- LLM não possui acesso direto ao Execution Adapter.
- Existe fallback seguro para erro de provider.

INSTRUÇÕES OBRIGATÓRIAS:
1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.
```

IMPLEMENTE SOMENTE A FASE 5.

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 5 DO AEGIS V1.2.

FASE:

AI / Decision Engine

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Forçar resposta LLM inválida.
- Verificar validação de confidence.
- Verificar ausência de candle futuro.
- Verificar logs/auditoria de modelo e prompt.
- Verificar que LLM não pode chamar broker.
- Verificar comportamento diante de timeout/erro.

CRITÉRIOS DE ACEITE:

- Provider LLM é abstraído por interface.
- Resposta inválida é rejeitada.
- Confidence é validada no intervalo definido.
- Prompt/model/version são auditáveis.
- Somente candles fechados entram no contexto.
- LLM não possui acesso direto ao Execution Adapter.
- Existe fallback seguro para erro de provider.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 06 — Risk Engine

Objetivo

Implementar a camada determinística que decide se uma decisão pode ou não ser transformada em intenção de ordem.

Visão Arquitetural

Risk Engine é independente do LLM. Hard limits nunca podem ser sobrescritos por uma decisão de IA. O resultado deve ser explicitamente APPROVED ou REJECTED.

Critérios de Aceite

- Limite de exposição é respeitado.
- Limite de tamanho de posição é respeitado.
- Stop obrigatório é validado conforme configuração.
- Limite de perda é aplicado.
- Kill switch bloqueia novas ordens.
- Toda rejeição possui reason code.
- Não existe bypass do Risk Engine.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 6 DO PROJETO AEGIS V1.2.

FASE:
Risk Engine

OBJETIVO:
Implementar a camada determinística que decide se uma decisão pode ou não
ser transformada em intenção de ordem.

REGRAS ARQUITETURAIS:
Risk Engine é independente do LLM. Hard limits nunca podem ser sobrescritos
por uma decisão de IA. O resultado deve ser explicitamente APPROVED ou REJECTED.

CRITÉRIOS DE ACEITE:
- Limite de exposição é respeitado.
- Limite de tamanho de posição é respeitado.
- Stop obrigatório é validado conforme configuração.
- Limite de perda é aplicado.
- Kill switch bloqueia novas ordens.
- Toda rejeição possui reason code.
- Não existe bypass do Risk Engine.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 6.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 6 DO AEGIS V1.2.

FASE:
Risk Engine

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Testar posição acima do limite.
- Testar exposição acima do limite.
- Testar ordem sem stop quando stop é obrigatório.
- Testar kill switch.
- Testar perda diária máxima.
- Verificar impossibilidade de bypass.

CRITÉRIOS DE ACEITE:

- Limite de exposição é respeitado.
- Limite de tamanho de posição é respeitado.
- Stop obrigatório é validado conforme configuração.
- Limite de perda é aplicado.
- Kill switch bloqueia novas ordens.
- Toda rejeição possui reason code.
- Não existe bypass do Risk Engine.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 07 — Portfolio & Accounting

Objetivo

Implementar posições, saldo, cash, P&L; realizado/não realizado, fills, custo médio e estado patrimonial.

Visão Arquitetural

Portfolio é derivado de eventos de execução e não da decisão do LLM. O accounting deve ser determinístico e reproduzível.

Critérios de Aceite

- Fills alteram posições corretamente.
- P&L; realizado é calculável.
- P&L; não realizado é calculável.
- Cash é reconciliável.
- Posições abertas possuem estado consistente.
- Accounting possui testes com casos conhecidos.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 7 DO PROJETO AEGIS V1.2.

FASE:
Portfolio & Accounting

OBJETIVO:
Implementar posições, saldo, cash, P&L realizado/não realizado,
fills, custo médio e estado patrimonial.

REGRAS ARQUITETURAIS:
Portfolio é derivado de eventos de execução e não da decisão do LLM.
O accounting deve ser determinístico e reproduzível.

CRITÉRIOS DE ACEITE:
- Fills alteram posições corretamente.
- P&L realizado é calculável.
- P&L não realizado é calculável.
- Cash é reconciliável.
- Posições abertas possuem estado consistente.
- Accounting possui testes com casos conhecidos.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 7.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 7 DO AEGIS V1.2.

FASE:

Portfolio & Accounting

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Executar compra e venda simples.
- Verificar custo médio.
- Verificar P&L realizado.
- Verificar P&L não realizado.
- Verificar reconciliação de cash.
- Verificar determinismo.

CRITÉRIOS DE ACEITE:

- Fills alteram posições corretamente.
- P&L realizado é calculável.
- P&L não realizado é calculável.
- Cash é reconciliável.
- Posições abertas possuem estado consistente.
- Accounting possui testes com casos conhecidos.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 08 — Paper Execution Simulator

Objetivo

Criar o Execution Adapter simulado para transformar ordens aprovadas pelo Risk Engine em fills de paper trading.

Visão Arquitetural

Execution Simulator implementa o mesmo contrato que um futuro broker adapter, mas sem conexão financeira real. Nenhuma ordem LIVE é permitida.

Critérios de Aceite

- Somente ordens aprovadas pelo Risk Engine são executáveis.
- Ordens possuem lifecycle explícito.
- Fills são registrados.
- Slippage configurável pode ser simulado.
- Falhas de execução são simuláveis.
- Não existe chamada a broker real.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 8 DO PROJETO AEGIS V1.2.

FASE:
Paper Execution Simulator

OBJETIVO:
Criar o Execution Adapter simulado para transformar ordens aprovadas pelo
Risk Engine em fills de paper trading.

REGRAS ARQUITETURAIS:
Execution Simulator implementa o mesmo contrato que um futuro broker adapter,
mas sem conexão financeira real. Nenhuma ordem LIVE é permitida.

CRITÉRIOS DE ACEITE:
- Somente ordens aprovadas pelo Risk Engine são executáveis.
- Ordens possuem lifecycle explícito.
- Fills são registrados.
- Slippage configurável pode ser simulado.
- Falhas de execução são simuláveis.
- Não existe chamada a broker real.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 8.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 8 DO AEGIS V1.2.

FASE:

Paper Execution Simulator

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Tentar executar ordem rejeitada pelo Risk Engine.
- Verificar lifecycle.
- Verificar fill.
- Verificar slippage.
- Verificar falha simulada.
- Procurar qualquer integração LIVE.

CRITÉRIOS DE ACEITE:

- Somente ordens aprovadas pelo Risk Engine são executáveis.
- Ordens possuem lifecycle explícito.
- Fills são registrados.
- Slippage configurável pode ser simulado.
- Falhas de execução são simuláveis.
- Não existe chamada a broker real.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 09 — Order Lifecycle

Objetivo

Implementar estados de ordem, idempotência, correlação e tratamento de transições.

Visão Arquitetural

Ordens devem possuir identificadores únicos e lifecycle explícito. Reprocessamento de evento não pode gerar duplicação de execução.

CrITÉrios de Aceite

- Order IDs são únicos.
- Transições inválidas são rejeitadas.
- Eventos duplicados são tratados de forma idempotente.
- Status final é consistente.
- Ordens possuem correlation ID.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 9 DO PROJETO AEGIS V1.2.

FASE:
Order Lifecycle

OBJETIVO:
Implementar estados de ordem, idempotência, correlação e tratamento de
transições.

REGRAS ARQUITETURAIS:
Ordens devem possuir identificadores únicos e lifecycle explícito.
Reprocessamento de evento não pode gerar duplicação de execução.

CRITÉRIOS DE ACEITE:
- Order IDs são únicos.
- Transições inválidas são rejeitadas.
- Eventos duplicados são tratados de forma idempotente.
- Status final é consistente.
- Ordens possuem correlation ID.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 9.
```

Prompt de Auditoria

```
VOCÊ É O AUDITOR DA FASE 9 DO AEGIS V1.2.
```

FASE:

Order Lifecycle

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Enviar evento duplicado.
- Tentar transição inválida.
- Verificar idempotência.
- Verificar correlation ID.
- Verificar consistência do status final.

CRITÉRIOS DE ACEITE:

- Order IDs são únicos.
- Transições inválidas são rejeitadas.
- Eventos duplicados são tratados de forma idempotente.
- Status final é consistente.
- Ordens possuem correlation ID.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 10 — Persistence & Event History

Objetivo

Persistir decisões, riscos, ordens, fills, posições, configurações e eventos de auditoria.

Visão Arquitetural

Eventos críticos devem ser rastreáveis. Persistência não deve alterar a semântica do domínio.

Critérios de Aceite

- Eventos críticos possuem timestamp.
- Decisões são vinculadas ao contexto.
- Ordens podem ser rastreadas até a decisão.
- Fills podem ser rastreados até a ordem.
- Migrations são versionadas.
- Dados essenciais sobrevivem ao restart.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCE É O CODER RESPONSÁVEL PELA FASE 10 DO PROJETO AEGIS V1.2.

FASE:
Persistence & Event History

OBJETIVO:
Persistir decisões, riscos, ordens, fills, posições, configurações e eventos
de auditoria.

REGRAS ARQUITETURAIS:
Eventos críticos devem ser rastreáveis. Persistência não deve alterar a
semântica do domínio.

CRITÉRIOS DE ACEITE:
- Eventos críticos possuem timestamp.
- Decisões são vinculadas ao contexto.
- Ordens podem ser rastreadas até a decisão.
- Fills podem ser rastreados até a ordem.
- Migrations são versionadas.
- Dados essenciais sobrevivem ao restart.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 10.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 10 DO AEGIS V1.2.

FASE:

Persistence & Event History

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Verificar cadeia Decision → Risk → Order → Fill.
- Reiniciar containers e verificar persistência.
- Verificar migrations.
- Verificar timestamps.
- Verificar ausência de registros órfãos.

CRITÉRIOS DE ACEITE:

- Eventos críticos possuem timestamp.
- Decisões são vinculadas ao contexto.
- Ordens podem ser rastreadas até a decisão.
- Fills podem ser rastreados até a ordem.
- Migrations são versionadas.
- Dados essenciais sobrevivem ao restart.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 11 — Orchestration

Objetivo

Implementar o fluxo operacional que conecta market data, decision, risk, execution, portfolio e persistência.

Visão Arquitetural

O orchestration layer coordena componentes, mas não deve conter regras de negócio críticas que pertençam ao domínio.

Critérios de Aceite

- Pipeline possui ordem determinística.
- Cada etapa pode ser observada.
- Falhas não geram execução silenciosa.
- Risk ocorre antes de Execution.
- Portfolio é atualizado após fill.
- Eventos são correlacionáveis.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

VOCÊ É O CODER RESPONSÁVEL PELA FASE 11 DO PROJETO AEGIS V1.2.

FASE:
Orchestration

OBJETIVO:
Implementar o fluxo operacional que conecta market data, decision, risk, execution, portfolio e persistência.

REGRAS ARQUITETURAIS:
O orchestration layer coordena componentes, mas não deve conter regras de negócio críticas que pertençam ao domínio.

CRITÉRIOS DE ACEITE:
- Pipeline possui ordem determinística.
- Cada etapa pode ser observada.
- Falhas não geram execução silenciosa.
- Risk ocorre antes de Execution.
- Portfolio é atualizado após fill.
- Eventos são correlacionáveis.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução. Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
 - arquivos criados/alterados;
 - testes executados;
 - critérios de aceite atendidos;
 - limitações encontradas;
 - qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 11.

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 11 DO AEGIS V1.2.

FASE:
Orchestration

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Simular falha em cada etapa.
- Verificar sequência.
- Verificar Risk antes de Execution.
- Verificar atualização pós-fill.
- Verificar ausência de lógica de negócio indevida no orchestrator.

CRITÉRIOS DE ACEITE:

- Pipeline possui ordem determinística.
- Cada etapa pode ser observada.
- Falhas não geram execução silenciosa.
- Risk ocorre antes de Execution.
- Portfolio é atualizado após fill.
- Eventos são correlacionáveis.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 12 — API & Dashboard

Objetivo

Construir a interface visual do Aegis para acompanhamento das operações, posições, P&L, eventos, configurações e estado do sistema.

Visão Arquitetural

Frontend consome API. Não possui regras críticas de risco. Configurações são validadas pelo backend.

Critérios de Aceite

- Dashboard exibe saldo.
- Dashboard exibe posições abertas.
- Dashboard exibe P&L;
- Dashboard exibe ordens e estados.
- Dashboard exibe eventos relevantes.
- Configuração de stop é validada no backend.
- Configuração LLM não expõe secrets.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 12 DO PROJETO AEGIS V1.2.

FASE:
API & Dashboard

OBJETIVO:
Construir a interface visual do Aegis para acompanhamento das operações,
posições, P&L, eventos, configurações e estado do sistema.

REGRAS ARQUITETURAIS:
Frontend consome API. Não possui regras críticas de risco. Configurações
são validadas pelo backend.

CRITÉRIOS DE ACEITE:
- Dashboard exibe saldo.
- Dashboard exibe posições abertas.
- Dashboard exibe P&L.
- Dashboard exibe ordens e estados.
- Dashboard exibe eventos relevantes.
- Configuração de stop é validada no backend.
- Configuração LLM não expõe secrets.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 12.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 12 DO AEGIS V1.2.

FASE:

API & Dashboard

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Verificar todas as telas previstas.
- Alterar configuração inválida.
- Verificar validação server-side.
- Verificar exposição de secrets.
- Verificar consistência de P&L com backend.

CRITÉRIOS DE ACEITE:

- Dashboard exibe saldo.
- Dashboard exibe posições abertas.
- Dashboard exibe P&L.
- Dashboard exibe ordens e estados.
- Dashboard exibe eventos relevantes.
- Configuração de stop é validada no backend.
- Configuração LLM não expõe secrets.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 13 — Configuration & LLM Providers

Objetivo

Centralizar configuração operacional e abstração dos providers de LLM, incluindo seleção de modelo, parâmetros, timeout e credenciais.

Visão Arquitetural

Secrets nunca são persistidos em texto puro no frontend. Providers são intercambiáveis e a configuração efetiva de uma decisão deve ser auditável.

Critérios de Aceite

- Provider pode ser selecionado por configuração.
- Credenciais não aparecem em logs.
- Timeout é configurável.
- Modelo utilizado é registrado.
- Configuração inválida é rejeitada.
- Provider indisponível não causa execução automática.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

VOCÊ É O CODER RESPONSÁVEL PELA FASE 13 DO PROJETO AEGIS V1.2.

FASE:
Configuration & LLM Providers

OBJETIVO:
Centralizar configuração operacional e abstração dos providers de LLM, incluindo seleção de modelo, parâmetros, timeout e credenciais.

REGRAS ARQUITETURAIS:
Secrets nunca são persistidos em texto puro no frontend. Providers são intercambiáveis e a configuração efetiva de uma decisão deve ser auditável.

CRITÉRIOS DE ACEITE:
- Provider pode ser selecionado por configuração.
- Credenciais não aparecem em logs.
- Timeout é configurável.
- Modelo utilizado é registrado.
- Configuração inválida é rejeitada.
- Provider indisponível não causa execução automática.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução. Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
 - arquivos criados/alterados;
 - testes executados;
 - critérios de aceite atendidos;
 - limitações encontradas;
 - qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 13.

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 13 DO AEGIS V1.2.

FASE:

Configuration & LLM Providers

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Verificar secret masking.
- Testar provider inexistente.
- Testar timeout.
- Verificar modelo registrado.
- Verificar comportamento em indisponibilidade.

CRITÉRIOS DE ACEITE:

- Provider pode ser selecionado por configuração.
- Credenciais não aparecem em logs.
- Timeout é configurável.
- Modelo utilizado é registrado.
- Configuração inválida é rejeitada.
- Provider indisponível não causa execução automática.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 14 — Observability, Audit & Safety

Objetivo

Consolidar logs estruturados, métricas, audit trail, kill switch, estado operacional e mecanismos de segurança.

Visão Arquitetural

Safety é transversal. O sistema deve ser capaz de explicar por que uma decisão foi tomada, por que foi aprovada/rejeitada e o que ocorreu depois.

Critérios de Aceite

- Eventos críticos possuem correlation ID.
- Logs não contêm secrets.
- Kill switch funciona.
- Falhas críticas são identificáveis.
- Decisão → Risk → Order → Fill é rastreável.
- Sistema possui estado operacional explícito.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

```
VOCÊ É O CODER RESPONSÁVEL PELA FASE 14 DO PROJETO AEGIS V1.2.

FASE:
Observability, Audit & Safety

OBJETIVO:
Consolidar logs estruturados, métricas, audit trail, kill switch,
estado operacional e mecanismos de segurança.

REGRAS ARQUITETURAIS:
Safety é transversal. O sistema deve ser capaz de explicar por que uma
decisão foi tomada, por que foi aprovada/rejeitada e o que ocorreu depois.

CRITÉRIOS DE ACEITE:
- Eventos críticos possuem correlation ID.
- Logs não contêm secrets.
- Kill switch funciona.
- Falhas críticas são identificáveis.
- Decisão → Risk → Order → Fill é rastreável.
- Sistema possui estado operacional explícito.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução.
Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
- arquivos criados/alterados;
- testes executados;
- critérios de aceite atendidos;
- limitações encontradas;
- qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 14.
```

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 14 DO AEGIS V1.2.

FASE:

Observability, Audit & Safety

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Acionar kill switch.
- Pesquisar vazamento de secrets.
- Rastrear uma operação ponta a ponta.
- Simular falha crítica.
- Verificar correlation IDs.
- Verificar logs estruturados.

CRITÉRIOS DE ACEITE:

- Eventos críticos possuem correlation ID.
- Logs não contêm secrets.
- Kill switch funciona.
- Falhas críticas são identificáveis.
- Decisão → Risk → Order → Fill é rastreável.
- Sistema possui estado operacional explícito.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 15 — End-to-End Validation

Objetivo

Executar validação integrada utilizando Replay e Paper Execution, cobrindo o caminho completo desde dados de mercado até portfolio.

Visão Arquitetural

Esta fase não introduz arquitetura nova. Seu objetivo é provar que os componentes previamente implementados funcionam juntos.

Critérios de Aceite

- Pipeline completo executa em Replay.
- Pipeline completo executa em Paper.
- Nenhuma ordem ultrapassa Risk.
- Portfolio reconcilia com fills.
- Replay permanece determinístico.
- Cenários de erro possuem comportamento seguro.
- Testes de integração são reproduzíveis.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceito.

Prompt de Implementação

VOCÊ É O CODER RESPONSÁVEL PELA FASE 15 DO PROJETO AEGIS V1.2.

FASE:
End-to-End Validation

OBJETIVO:
Executar validação integrada utilizando Replay e Paper Execution, cobrindo o caminho completo desde dados de mercado até portfolio.

REGRAS ARQUITETURAIS:
Esta fase não introduz arquitetura nova. Seu objetivo é provar que os componentes previamente implementados funcionam juntos.

CRITÉRIOS DE ACEITE:

- Pipeline completo executa em Replay.
- Pipeline completo executa em Paper.
- Nenhuma ordem ultrapassa Risk.
- Portfolio reconcilia com fills.
- Replay permanece determinístico.
- Cenários de erro possuem comportamento seguro.
- Testes de integração são reproduzíveis.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução. Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
 - arquivos criados/alterados;
 - testes executados;
 - critérios de aceite atendidos;
 - limitações encontradas;
 - qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 15.

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 15 DO AEGIS V1.2.

FASE:
End-to-End Validation

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- Executar cenário nominal.
- Executar cenário de perda.
- Executar cenário de limite de exposição.
- Executar kill switch.
- Executar provider LLM indisponível.
- Comparar dois replays idênticos.
- Verificar cadeia completa de auditoria.

CRITÉRIOS DE ACEITE:

- Pipeline completo executa em Replay.
- Pipeline completo executa em Paper.
- Nenhuma ordem ultrapassa Risk.
- Portfolio reconcilia com fills.
- Replay permanece determinístico.
- Cenários de erro possuem comportamento seguro.
- Testes de integração são reproduzíveis.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

FASE 16 — Certification & Release Gate

Objetivo

Certificar o estado final da V1.2. Esta fase NÃO implementa novas funcionalidades.

Visão Arquitetural

A certificação compara a implementação real com o Blueprint V1.2, contratos, critérios de aceite, testes e invariantes. Qualquer divergência bloqueia o release.

Critérios de Aceite

- Todos os testes obrigatórios passam.
- Todos os gates anteriores estão aprovados.
- Não existem blockers arquiteturais abertos.
- LIVE execution permanece desabilitado.
- Replay e Paper funcionam.
- Critérios críticos de segurança possuem testes.
- Artefato final é identificado como PAPER_READY.

Gate: a fase só pode avançar após auditoria PASS ou PASS_WITH_WARNINGS formalmente aceite.

Prompt de Implementação

VOCÊ É O CODER RESPONSÁVEL PELA FASE 16 DO PROJETO AEGIS V1.2.

FASE:
Certification & Release Gate

OBJETIVO:
Certificar o estado final da V1.2. Esta fase NÃO implementa novas funcionalidades.

REGRAS ARQUITETURAIS:
A certificação compara a implementação real com o Blueprint V1.2, contratos, critérios de aceite, testes e invariantes. Qualquer divergência bloqueia o release.

CRITÉRIOS DE ACEITE:
- Todos os testes obrigatórios passam.
- Todos os gates anteriores estão aprovados.
- Não existem blockers arquiteturais abertos.
- LIVE execution permanece desabilitado.
- Replay e Paper funcionam.
- Critérios críticos de segurança possuem testes.
- Artefato final é identificado como PAPER_READY.

INSTRUÇÕES OBRIGATÓRIAS:

1. Não altere decisões arquiteturais do Blueprint.
2. Não introduza tecnologias fora da stack congelada.
3. Não implemente funcionalidades de fases futuras.
4. Não crie integração LIVE.
5. Não coloque lógica crítica de Risk no frontend.
6. Não permita bypass do Risk Engine.
7. Escreva testes automatizados para o comportamento implementado.
8. Mantenha o código modular e testável.
9. Se encontrar uma contradição arquitetural, NÃO invente uma solução. Pare e reporte ARCHITECTURAL_BLOCKER.
10. Ao terminar, informe:
 - arquivos criados/alterados;
 - testes executados;
 - critérios de aceite atendidos;
 - limitações encontradas;
 - qualquer ARCHITECTURAL_BLOCKER.

IMPLEMENTE SOMENTE A FASE 16.

Prompt de Auditoria

VOCÊ É O AUDITOR DA FASE 16 DO AEGIS V1.2.

FASE:
Certification & Release Gate

VOCÊ NÃO DEVE IMPLEMENTAR FUNCIONALIDADES.

AUDITE ESPECIFICAMENTE:

- NÃO implementar funcionalidades.
- Executar toda a suíte de testes.
- Verificar todos os gates.
- Verificar ausência de broker LIVE.
- Verificar invariantes de Risk.
- Verificar determinismo do Replay.
- Produzir relatório final PASS/FAIL.

CRITÉRIOS DE ACEITE:

- Todos os testes obrigatórios passam.
- Todos os gates anteriores estão aprovados.
- Não existem blockers arquiteturais abertos.
- LIVE execution permanece desabilitado.
- Replay e Paper funcionam.
- Critérios críticos de segurança possuem testes.
- Artefato final é identificado como PAPER_READY.

REGRAS DE AUDITORIA:

1. Leia o código realmente existente.
2. Não aceite alegações do coder sem evidência.
3. Execute os testes relevantes.
4. Crie testes temporários se necessário para provar uma violação.
5. Verifique violações arquiteturais.
6. Verifique se funcionalidades futuras foram indevidamente implementadas.
7. Verifique ausência de bypass de segurança.
8. Verifique ausência de secrets hardcoded.
9. Não corrija o código durante a auditoria.
10. Classifique o resultado como:

PASS

PASS_WITH_WARNINGS

FAIL

ARCHITECTURAL_BLOCKER

FORMATO DO RELATÓRIO:

STATUS:

EVIDÊNCIAS:

TESTES EXECUTADOS:

CRITÉRIOS ATENDIDOS:

CRITÉRIOS NÃO ATENDIDOS:

VIOLAÇÕES ARQUITETURAIS:

RISCOS:

RECOMENDAÇÃO:

7. Modelo de Gates

Status	Significado
PASS	Todos os critérios atendidos.
PASS_WITH_WARNINGS	Critérios atendidos; warnings não bloqueantes documentados.
FAIL	Critério funcional ou de aceite não atendido.
ARCHITECTURAL_BLOCKER	Violação/ambiguidade arquitetural que exige decisão antes de continuar.

Regra de progressão: **FAIL e ARCHITECTURAL_BLOCKER** bloqueiam a próxima fase. O coder deve corrigir a fase atual e submetê-la novamente à auditoria.

8. Determinismo e Reprodutibilidade

- O Replay deve produzir a mesma sequência de eventos quando executado com a mesma entrada.
 - Configuração do replay deve ser registrada.
 - Modelo, provider e versão de prompt devem ser identificáveis para decisões de IA.
 - Dados de mercado devem possuir origem e versão identificáveis.
 - A execução Paper deve possuir parâmetros de simulação rastreáveis.
- O objetivo não é tornar um LLM necessariamente determinístico em todos os cenários, mas tornar o experimento rastreável e permitir identificar exatamente quais entradas, configurações e versões produziram uma decisão.

9. Governança do Uso de LLM

- O Blueprint pode ser utilizado por agentes LLM para implementação, porém o código não é considerado correto apenas porque um LLM afirma que está correto.
- Contratos tipados.
 - Schemas de validação.
 - Testes automatizados.
 - Invariantes de Risk.
 - Testes de Replay.
 - Auditorias específicas por fase.
 - Evidence-based acceptance.

A fonte de verdade é o conjunto **Blueprint + contratos + testes + invariantes + evidências de execução**, e não a opinião do coder ou do auditor.

10. Configurações Principais da V1.2

Grupo	Exemplos
Trading	Capital inicial, ativos, timeframe, limites de exposição
Risk	Stop Loss, posição máxima, perda máxima, kill switch
LLM	Provider, modelo, temperature, timeout, prompt version
Replay	Dataset, período, seed, modo determinístico
Paper	Slippage, taxas, latência simulada
System	Logs, métricas, timezone, ambiente

11. Fluxo Operacional Canônico

1. Market Data fornece candle fechado.
2. Context Builder prepara os dados permitidos.
3. AI Decision Engine analisa o contexto.
4. Decision Contract valida a saída.
5. Risk Engine verifica todos os limites.
6. Se REJECTED, registra o motivo e encerra o fluxo.
7. Se APPROVED, cria Approved Order Intent.
8. Execution Adapter Paper simula a execução.
9. Fill é registrado.
10. Portfolio atualiza posição e P&L.;
11. Audit Trail registra o fluxo completo.
12. Dashboard apresenta o estado atual.

12. Critérios de Release V1.2

- Todas as 16 fases possuem gate.
- Todos os testes obrigatórios passam.
- Replay funciona de forma determinística.
- Paper Trading funciona ponta a ponta.
- Risk Engine bloqueia violações.
- Kill switch funciona.
- Dashboard apresenta estado operacional consistente.
- Decisões LLM são rastreáveis.
- Nenhum secret está exposto.
- Nenhuma integração LIVE está habilitada.
- Não existem ARCHITECTURAL_BLOCKERS abertos.

Release State: PAPER_READY

13. V1.2 — Decisões Incorporadas

Decisão	Implementação na V1.2
Replay antecipado	Replay passou a ser fundação de validação, não funcionalidade tardia.
Stack congelada	Python 3.12+, FastAPI, React/TypeScript, PostgreSQL, Docker e demais componentes definidos.
Certificação	Fase 16 deixou de ser implementação e passou a ser Release Gate.
Auditorias	Cada fase recebeu critérios específicos de auditoria.
Aceite mensurável	Critérios passaram a ser verificáveis através de testes e evidências.
Governança LLM	Contratos, invariantes e testes reduzem a circularidade LLM → LLM.
Escopo acadêmico	Removido completamente.
LIVE	Explicitamente fora da V1.2.
Modos	REPLAY e PAPER são os modos funcionais da V1.2.
Número de fases	16 fases, sem artificialidade de 20 fases.

AEGIS V1.2 — ARCHITECTURE FROZEN

A partir deste artefato, alterações arquiteturais não devem ser realizadas durante a implementação da V1.2.

Uma alteração estrutural necessária deve gerar uma nova revisão formal do Blueprint, preservando a rastreabilidade da versão.

V1.2 RELEASE TARGET: PAPER_READY

LIVE EXECUTION: OUT OF SCOPE

AEGIS — Autonomous Trading Intelligence System